The Price of Local Ordering in Two-Stage Assembly Flow Shops

Annotated Edition for Executives and Practitioners

Manuscript prepared for journal submission (Paper C) — not yet submitted or peer reviewed

Eric Bowman
King, Berlin, Germany
eric.bowman@king.com
ORCID: 0009-0003-9171-7217

How to read this document: the left column is the manuscript, word for word. The boxes in the right column are plain-language commentary for readers who are not scheduling theorists. The manuscript itself contains none of the boxes.

Abstract

In the divisible-work two-stage assembly flow shop, permutation schedules—one job order shared by every machine—dominate pointwise, so adopting a shared order is never worse. This paper quantifies the converse: how much can abandoning the shared order cost? In the fluid homogeneous model we first pin the optimum exactly: every schedule satisfies $k\,C_{\max} \geq W + w_{\max}$, and the synchronized schedule attains it under *every* job order; consequently, under arbitrarily distributed random work, a fixed shared order—chosen with no foresight—stochastically dominates every scheduling policy on makespan, and the permutation class is stochastically complete for all regular objectives. Against this exact baseline the price of local ordering is sharp: arbitrary per-machine orders combined with a downstream stage that may hold capacity idle for a preferred job cost exactly $2W/(W + w_{\max}) \to 2$ on makespan, while *any* work-conserving downstream stage—regardless of its processing order—caps the price at $2 - 1/m$, where m is the number of stage-1 machines; both constants are tight. The same dichotomy governs total flowtime, with tight constants 3 and $3 - 2/m$; for weighted completion time no constant-factor ceiling exists. The structural conclusion is that the expensive downstream behavior is *work-withholding*, not misordering: a stage that idles while released work waits forfeits as much again as all upstream misalignment combined. Every constant is verified by exhaustive enumeration over the instance family of the companion paper, where the worst cases match the theory exactly, and the two bracketing results are formalized in Lean 4.

What this paper settles, in one sentence: the earlier papers proved that putting every team on one shared priority list is *never worse*; this one proves how much it costs *not* to — and the price list is exact. Delivery date: up to twice as late, never more. Average delivery: up to three times the wait. Value delivery: no limit at all.

The headline that changes behavior: the worst downstream behavior is not working on things in the "wrong" order. It is *waiting* — a stage that sits idle because the item its priority list says to do next hasn't arrived yet, while other finished work piles up in front of it. That single behavior costs as much as every upstream coordination failure combined.

And one free lunch: for the date the *whole* portfolio lands, any shared list is mathematically optimal — even a wrong one, even when nobody can estimate the work. The list has to be *shared* to land everything soonest; it has to be *right* to land the *value* soonest.

1 Introduction

Dominance theorems are one-sided. For the two-stage assembly flow shop with divisible work—m parallel stage-1 machines of k identical workers feeding a single assembly stage of k workers—for every schedule there is a *permutation schedule*, with one job order shared by all stage-1 machines, that completes every job at the assembly stage no later (Bowman, 2026a). A companion paper maps where such dominance survives on deeper stage structures (Bowman, 2026b). These results say that coordinating on a shared order is *never worse*. They are silent on the question any adopter of the policy actually asks: how much worse is it *not* to coordinate?

The setting, in office language: several teams ("stage-1 machines") each have to do their part of every initiative ("job"), and a downstream integration stage can only start an initiative once *all* teams have finished their parts. Paper A proved the shared list is free insurance. But "never worse" doesn't tell a leadership team what the status quo — every team running its own list — actually costs. That's the question here.

This paper answers that question exactly, in the fluid (real-divisible) homogeneous form of the model. Three groups of results.

First, the baseline is the true optimum (Section 3). Every schedule—any per-machine orders, any work divisions, any downstream discipline, preemptive or not, idle or not—satisfies

$$k\,C_{\max} \;\geq\; W + w_{\max},$$

where W is the total work and $w_{\max}$ the largest job, and the synchronized schedule attains this bound under *every* shared order π (Theorem 2). Two consequences follow. Under arbitrarily distributed random work—any joint law—a fixed shared order, chosen before any work is known, is pathwise makespan-optimal, hence stochastically dominant over every policy including clairvoyant ones: for the date the whole portfolio of jobs lands, foresight is worth nothing and the choice of shared order is worth nothing; only *sharedness* matters (Corollary 4). And the pointwise dominance theorem of Bowman (2026a) couples, realization by realization, into a stochastic completeness statement: no policy beats the permutation class on any regular objective in any stochastic sense (Theorem 5). All prices below are therefore measured against the exact optimum, not against a possibly suboptimal synchronized baseline.

Why this part matters: before you can say "not coordinating costs 40%," you need to know 40% *of what*. This section proves the shared-list schedule isn't just good — it is *the* mathematical optimum for the portfolio completion date, and (the surprise) it doesn't matter which shared order you pick. Two practical readings: (1) every price in this paper is measured against perfection, not against a soft baseline; (2) arguing about the *order* of the list delays the portfolio by exactly zero — the order matters for *value* (later in the paper), not for the finish date.

"Stochastically dominates every policy including clairvoyant ones" means: even a scheduler who can see the future — who knows exactly how big every piece of work will turn out to be — cannot beat a shared list that was fixed before anything was known. For the portfolio date, estimates are worthless and foresight is worthless. That is why the operating rule can be simple.

Second, the price of local ordering on makespan is an exact dichotomy (Section 4). If every stage-1 machine chooses its own order and the downstream stage may *withhold* work—idle while released work waits, as a stage does when it insists on processing in a preferred order whose next job has not yet arrived—the worst-case makespan ratio is exactly $2W/(W + w_{\max})$, approaching 2 (Theorem 7). If instead the downstream stage is *work-conserving*—never idle while released work is unfinished, but otherwise free to process in any order whatsoever—the price drops to $2 - 1/m$, and this is tight even against a clairvoyant downstream (Theorem 9). The proof of the upper bound never uses the downstream *order*; only non-idleness. The structural conclusion, which we believe is the practically important one, is that for makespan the expensive downstream sin is withholding, not misordering: a downstream stage that holds capacity idle for its preferred job forfeits as much again as all upstream misalignment combined.

The dichotomy, concretely. Picture a release gate that "respects the stack rank": initiative #7 is ready and waiting, but #3 is ranked higher and isn't here yet, so the gate waits for #3. That waiting — not the ranking — is what the math indicts. With two upstream teams: teams running their own lists costs at most $+50\%$ on the portfolio date *if the gate keeps pulling whatever is ready*; let the gate wait for its preferred item and the worst case doubles to $+100\%$. The factor splits exactly in half: upstream misalignment owns one half, downstream waiting owns the other.

Three currencies, three prices. Portfolio date: capped at $2\times$. Average initiative wait: capped at $3\times$. Value delivered: *uncapped* — if one initiative carries most of the value and teams order it locally, the value delay grows without limit. The more concentrated your value, the more local ordering costs you.

Third, the same dichotomy governs flowtime, and no ceiling exists for weighted completion time (Sections 5–6). For equal works the total-flowtime price is exactly $(3n + 1)/(n + 3) \to 3$ under full anarchy and $3-2/m$ under a work-conserving downstream, both tight. For weighted completion time the price is unbounded: concentrating value on one small job and ordering it locally multiplies the weighted objective by $\Theta(W/w_{\max})$.

Every constant above is exact at finite sizes in a way that can be checked by brute force, and we have checked it: over the 39-configuration exhaustive enumeration family of Bowman (2026a) (roughly 4.0×10^6 schedules), with the unsynchronized population split by downstream discipline, the worst cases agree with the theory to the digit, including two structural predictions — the two downstream classes must coincide once $m \geq n$, and the withholding class must be empty at $n = 2$ (Section 7). The two bracketing results (Theorem 2's lower bound, and the universal upper bound behind Theorem 7) are additionally formalized in Lean 4 with no unproven steps.

Why brute force? Every theorem here predicts exact worst-case numbers at small sizes — not asymptotic tendencies. So the theory can be *falsified* by enumerating every possible schedule of small instances and checking whether any schedule beats the predicted bounds or fails to reach them. About four million schedules were enumerated; every worst case landed exactly on the predicted fraction, and two structural side-predictions (cases where two categories must merge, and a case where one category cannot exist at all) came out exactly as required. "Formalized in Lean 4" means the two load-bearing inequalities were additionally re-proved inside a proof assistant — software that checks every logical step mechanically, so a subtle human error cannot survive.

1.1 Related work

The question belongs to the price-of-anarchy tradition of Koutsoupias and Papadimitriou (1999): compare a coordinated optimum against the

Placing the result. "Price of anarchy" is a standard lens from game theory: how bad can things get when nobody coordinates? Here there's no game — just teams with separate to-do lists — but the comparison is the same. The deepest connec-

worst outcome of uncoordinated choices. Our setting differs from game-theoretic scheduling in that we do not model incentives or equilibria; "anarchy" here is simply the absence of a shared order, which is how the phenomenon presents in the motivating applications (portfolios of cross-team initiatives; multi-stage processing pipelines). The two-stage assembly flow shop originates with Lee et al. (1993) and Potts et al. (1995), who proved permutation dominance and approximation results in the unit-capacity case; see Hariri and Potts (1997) for exact methods and Bowman (2026a) for the divisible-work generalization and the literature between.

The constant $2 - 1/m$ of Theorem 9 has the same form as Graham's bound for list scheduling on m identical machines (Graham, 1966; Graham et al., 1979), and the resemblance is not coincidental: both results bound the damage of arbitrary orderings by a busy-period argument in which work conservation is the only property used. The settings are different—Graham's m counts parallel machines executing the jobs, our m counts upstream machines feeding an assembly stage, and our ratio is measured against an exactly known optimum—but the mechanism, that non-idleness alone buys a factor-$(2 - 1/m)$ guarantee, appears to be the same phenomenon in two guises. The stochastic results connect to the classical observation that simple index policies retain optimality under uncertainty (Rothkopf, 1966; Smith, 1956; Pinedo, 2016); Corollary 4 is an extreme instance in which the optimal policy requires no distributional information at all. Divisible (malleable) work is surveyed by Drozdowski (2009).

tion is to a 1966 result of Graham: on m parallel machines, *any* greedy schedule — never idle while work waits — is within a factor $2 - 1/m$ of optimal, no matter how badly ordered. The same constant appears here for a different reason in a different place (our m counts the *feeding* teams, not the working machines), but the underlying physics is identical: *keeping busy* is worth a factor of two; cleverness about order buys the rest. Sixty years apart, the same moral.

2 Model and policy classes

There are n jobs; job j requires $w_j > 0$ units of work on each of m stage-1 machines and w_j units on a single stage-2 (assembly) machine. Every machine has k identical unit-rate workers. Work is divisible: a machine that devotes all k workers to one job processes it at rate k. Write $W = \sum_j w_j$ and $w_{\max} = \max_j w_j$. The *assembly constraint*: job j may not be processed at stage 2 before all m stage-1 machines have completed it; the time this happens is j's *availability* a_j. All jobs are present at time zero. This is the homogeneous fluid form of the model of Bowman (2026a); the integer-divisible form used there for the pointwise theorem appears here only in Theorem 5 and in the computational section.

Translation table. "Machine" = team. "k workers, divisible work" = the team can swarm one item and finish it k times sooner — the model where everyone pulls the top item together. "Availability" = the moment the *last* team finishes its part, which is when integration can begin — the fork-join bottleneck this whole series is about. "Team-sequential" = each team works one item at a time, at full strength, and upstream teams never sit idle. The model deliberately gives the *uncoordinated* org every benefit of the doubt: its teams work just as hard — they only order differently.

A schedule is *team-sequential* if every machine processes one job at a time at full rate k, stage-1 machines without inserted idleness. (Stage-1 idleness and unbalanced work divisions can only increase availabilities and are briefly discussed after each upper bound; the lower bound of Theorem 2 holds with no restriction at all.) Within team-sequential schedules we distinguish the downstream discipline:

Definition 1 (Policy classes)**.** *A team-sequential schedule is:*

- synchronized *if all stage-1 machines and the stage-2 machine process the jobs in one common order π;*
- work-conserving downstream *if the stage-2 machine is never idle while some released job is unfinished—its processing order is otherwise arbitrary;*
- work-withholding downstream *if it processes the jobs back-to-back in some fixed preferred order, each job starting as soon as the stage is free* and *the job is available—so the stage idles exactly when its order's next job has not yet arrived, even while other released work waits. (This is the behavior of a stage that enforces a priority list internally; it is the only inserted idleness we allow downstream, and the enumeration model of Section 7 matches it.)*

The three behaviors, in office language. *Synchronized*: everyone, including the integration stage, works the same list. *Work-conserving*: the integration stage takes whatever has arrived — any order, even a silly one — but never sits idle while finished work waits. *Work-withholding*: the integration stage has its own list and *waits* for the next item on it, letting ready work queue up. Note what withholding is in real life: it is what "enforce the stack rank at every stage" *operationally means* whenever the top-ranked item hasn't arrived yet.

The three scoreboards. Makespan = when the *last* initiative lands (the portfolio date). Flowtime = the sum of all landing dates (equivalently, the average wait). Weighted completion = landing dates weighted by how much each initiative is worth (time to value).

Upstream anarchy *means the stage-1 machines use arbitrary, possibly different orders.* Full anarchy *means upstream anarchy with a work-withholding downstream.*

Objectives are functions of the stage-2 completion vector $(C_j)_j$: makespan $C_{\max} = \max_j C_j$, total flowtime $\sum_j C_j$, and weighted completion time $\sum_j v_j C_j$ for non-negative values v_j.

3 The exact optimum, deterministically and stochastically

Theorem 2 (The optimum, and its order-independence)**.**

(a) Every schedule—team-sequential or not, with arbitrary work divisions, inserted idleness, and downstream discipline, preemptive or not—satisfies $k\,C_{\max} \geq W + w_{\max}$.

(b) For every order π*, the synchronized team-sequential schedule with order* π *has* $C_{\max} = (W + w_{\max})/k$.

Hence the optimal makespan is exactly $(W + w_{\max})/k$*, attained by the shared order—*any *shared order.*

Reading the formula. W/k is the bare minimum — all the work, done at full speed. The extra $w_{\max}/k$ is the price of the hand-off: the biggest item must first be finished upstream and *then* integrated, so its own size is paid twice and at least one of those payments cannot be hidden behind other work. Part (a) says *nobody* — any schedule whatsoever — can beat $(W+w_{\max})/k$. Part (b) says the shared list achieves it exactly, *whatever the order*: the pipeline never starves, so the only unavoidable wait is that one hand-off.

Proof. (a) Let M be a job of maximal work and let $a = a_M$ be its availability; fix one stage-1 machine that completes M at time a (if several, any). By time a that machine has processed at most ak units of work, of which $w_{\max}$ belong to M. A job is eligible for stage 2 only when complete on *all* machines, in particular on this one, so the total work of jobs other than M available by time a is at most $ak - w_{\max}$, and stage 2 can have completed at most that much by time a. At least $W - (ak - w_{\max})$ units of stage-2 work remain at time a, processed at rate at most k:

$$C_{\max} \;\geq\; a + \frac{W - ak + w_{\max}}{k} \;=\; \frac{W + w_{\max}}{k},$$

independent of a. The argument uses only capacity counting; it is indifferent to work divisions, idleness, preemption, and the number of machines.

(b) Under the shared order π, stage 1 completes the job at position s at time S_s/k where $S_s = \sum_{t \leq s} w_{\pi(t)}$. Writing F_s for its stage-2 completion and $M_s = \max_{t \leq s} w_{\pi(t)}$, induction on s gives $F_s = (S_s + M_s)/k$: indeed $F_s = \max(S_s, S_{s-1} + M_{s-1})/k + w_{\pi(s)}/k = (S_{s-1} + \max(M_{s-1}, w_{\pi(s)}))/k + w_{\pi(s)}/k = (S_s + M_s)/k$. At $s = n$ this is $(W + w_{\max})/k$ for every π. □

The proof in one breath (part a). Look at the moment a when the biggest item finally clears its slowest team. That team has had capacity $a \cdot k$ so far, and $w_{\max}$ of it went into the big item — so everything *else* that could possibly have reached integration by now is at most $ak - w_{\max}$. Whatever integration hasn't done yet — at least $W - (ak - w_{\max})$ — still takes its time at rate k. Add it up and a cancels out: the bound holds no matter when the big item lands. It's pure bookkeeping — which is why no schedule, however clever, can evade it.

(part b) is a domino computation: with one shared list, by the time integration finishes item $s-1$, item s has always already arrived — except for the single biggest item seen so far, whose size is the only gap that ever opens. The formula $(S_s + M_s)/k$ says exactly that: cumulative work plus one biggest-item allowance.

Remark 3 (Lean formalization)**.** *Part (a) is machine-checked in Lean 4 over the concrete integer-divisible machine model of Bowman (2026a), with stage 2 abstracted by a cumulative-progress structure whose axioms (progress capped by the work, zero before release, total rate at most* k*) are satisfied by every physical execution; the formal statement,* $k\,T \geq W + w_M$ *for* every *job* M*, is correspondingly stronger than (a). The formalization also revealed that the choice of machine in the proof is immaterial: any machine bounds the released work, because every completion lies below the availability. The development has no unproven steps and uses only the standard axioms.*

What the proof assistant added. Formalizing isn't just belt-and-braces: the machine-checked version came out *stronger* (it holds with any job in the role of "the big one") and *simpler* (the human proof picked a special machine; the formal proof showed any machine works). When mechanical checking simplifies a proof, that's evidence the result is robust rather than delicate.

The optimum being attained by every shared order has a stochastic consequence that we state in full generality.

Corollary 4 (A fixed shared order is stochastically optimal for makespan)**.** *Let the works* $(W_1, \ldots, W_n)$ *be random with an arbitrary joint distribution, and fix any order* π *in advance. Then, pathwise, the synchronized*

The estimates objection, retired. The standard objection to any scheduling policy is "our estimates are terrible." This corollary says: for the portfolio date, estimates are *irrelevant*. Fix any shared order before knowing anything; however the work

schedule with order π achieves $C_{\max} = (W + W_{\max})/k \le C_{\max}(S)$ for every scheduling policy S, including policies with full knowledge of the realization. In particular the fixed shared order stochastically dominates every policy on makespan.

Proof. Apply Theorem 2 realization by realization: (b) gives the value of the fixed-order schedule, (a) bounds every competitor below by the same value. □

sizes turn out — any distribution, any correlations — that schedule ties or beats every alternative, *including a scheduler who knew all the sizes in advance*, on every single realization. Not on average: every time.

For objectives beyond makespan a single fixed order no longer suffices, but the *class* of shared-order schedules remains unbeatable, again under arbitrary randomness. The following statement is in the integer-divisible model of Bowman (2026a), whose pointwise theorem it couples; it is the formal sense in which all prices in this paper are measured against an optimum that stochastic work cannot dislodge.

The broader insurance policy. Beyond the portfolio date, you might care about average wait, value-weighted delivery, or any other sane measure. This theorem says: whatever you measure and however random the work is, nothing outside the shared-order family can win. Searching for a better policy may stop at the family's door. The fine print: *which* shared order is best for value may depend on how things turn out — the family is unbeatable, a single fixed member of it isn't (except for the date). Whether "order by value" is the best fixed choice on average is flagged as an open problem.

Theorem 5 (Stochastic completeness of the permutation class)**.** *Let the works have an arbitrary joint distribution and let S be any scheduling policy, possibly randomized and clairvoyant. There is a works-measurable permutation-schedule policy S^* with $C_j(S^*) \le C_j(S)$ for every job j, almost surely. Consequently, for every coordinatewise-monotone objective f, $f(C(S^*)) \le f(C(S))$ almost surely, and no policy beats the permutation class in expectation, in any quantile, or in the usual stochastic order, for any regular objective.*

Proof. The construction of Bowman (2026a) maps each realization of the works and of S's schedule to a permutation schedule dominating it pointwise; the permutation used is obtained by sorting the realized availabilities, so the map is measurable (there are finitely many orders, and completion times are measurable in the works). Composing S with this map yields S^*; the domination is pointwise on the same probability space, i.e. a coupling, and all claimed comparisons follow. □

Remark 6. *Theorem 5 is existential in the order, which may depend on the realization. Corollary 4 shows that for makespan the dependence vanishes. Whether a fixed* value-ordered *permutation is optimal in expectation for weighted completion time under exchangeable works—the natural analogue of weighted-shortest-expected-processing-time results (Rothkopf, 1966)—is open; see Section 8.*

4 The price of local ordering: makespan

How you actually double a schedule. The worst case is mundane: one team happens to do initiative J *last*; the integration stage's own list happens to put J *first*. So integration waits for everything upstream to finish before it starts anything at all — then does all the integration work in a second, fully serial act. Total: all the work, twice over. The matching upper bound says this is as bad as it gets: since no team ever idles, everything is upstream-finished by W/k, and integration takes at most another W/k. Doubling is the worst case *and* the ceiling.

Note the two necessary ingredients: a team that buried the item, *and* a gate that waited for it. Remove either one and the disaster doesn't happen — that's the next theorem.

Theorem 7 (Full anarchy doubles the schedule, exactly)**.** *Over team-sequential schedules with upstream anarchy and a work-withholding downstream,*

$$\sup_{\text{instances, schedules}} \frac{C_{\max}}{(W + w_{\max})/k} = \sup_{\text{instances}} \frac{2W}{W + w_{\max}} = 2,$$

with the inner supremum attained for every instance with $m \ge 2$: no team-sequential schedule exceeds $2W/k$, and for every instance some unsynchronized schedule reaches it.

Proof. Upper bound. Stage-1 machines are work-conserving and process at rate k, so every machine finishes all its work at W/k; hence $a_j \le W/k$ for every j. A downstream stage that processes its jobs back-to-back in any order σ, waiting only for availabilities, satisfies the max-plus recursion $F_{\sigma(s)} = \max(F_{\sigma(s-1)}, a_{\sigma(s)}) + w_{\sigma(s)}/k$, whence by induction $C_{\max} \le \max_j a_j + W/k \le 2W/k$.

Lower bound. Fix any job J. Let stage-1 machine 1 process J last (so $a_J = W/k$) and machine 2 use any different order (so the schedule is unsynchronized). Let the downstream stage process J *first*: it idles

until W/k, then has all W units to do with no further idleness, finishing at exactly $2W/k$. Against the optimum $(W + w_{\max})/k$ of Theorem 2 the ratio is $2W/(W + w_{\max})$, which tends to 2 as $w_{\max}/W \to 0$. □

Remark 8. *The upper bound is also machine-checked in Lean 4 at the release-time interface (makespan at most the largest availability plus the total downstream work, for every downstream order), along with the availability bound and computed witnesses for the smallest anti-aligned instance. With unbalanced work divisions the* lower *bound only improves for the adversary; the sweep of Section 7 shows split badness stacking on top of the ordering price exactly where divisions are not forced balanced.*

The protection theorem. Suppose the integration stage simply refuses to idle: whenever *anything* finished is waiting, it works on it — order be damned. Then no matter how badly the upstream teams misalign, the portfolio date is within $2-1/m$ of optimal: $+50\%$ worst case with two teams, $+67\%$ with three, approaching $+100\%$ only as the number of misaligned teams grows without bound. And "tight" cuts both ways: there are upstream misalignments so bad that *no* downstream cleverness — not even a psychic gate — can do better than $2 - 1/m$. The cap is real and the cap is exact.

The construction above needs the downstream stage to *wait* for J. The next theorem shows this is the only way to get anywhere near a factor of two.

Theorem 9 (Work conservation caps the price at $2-1/m$)**.** *Over team-sequential schedules with upstream anarchy and ANY work-conserving downstream discipline,*

$$C_{\max} \;\le\; \Big(2 - \frac{1}{m}\Big)\frac{W}{k} + \frac{w_{\max}}{k},$$

and the constant is tight: for equal works w and $m \mid n$ there are stage-1 orders under which every *downstream discipline—work-conserving, withholding, or clairvoyant—has $C_{\max} \ge (2-1/m)\,nw/k + w/k$. Hence the price of upstream anarchy under a work-conserving downstream is exactly $2 - 1/m$ asymptotically.*

Upper bound in one breath. Look at the last moment the gate is idle. Idle-but-work-conserving means: everything that has arrived is already integrated, and the gate is waiting only because some item is still stuck upstream. But every team has been working flat out the whole time — so each team's "not yet finished" pile is small, and an item is missing only if it's in *somebody's* pile. Adding up the m piles (that's the inclusion–exclusion line) shows there can't be much missing work this late — so the final busy stretch is short. Idleness late is impossible; idleness early is harmless. Both cases land on exactly $2-1/m$.

Lower bound: the rotation. Give the m teams the same circular list, each starting at a different point — like a round of singing. Then *every* initiative is near the end of *some* team's list, so nothing at all is finished everywhere until the last $1/m$-th of the upstream phase. Integration then faces all the work with almost no head start. No gate, however clairvoyant, can recover — the damage was done upstream. (This is also a useful warning: upstream misalignment that looks "balanced" — everyone using the same list, just offset — is precisely the worst kind.)

Proof. Upper bound. Let t_1 be the last instant stage 2 is idle before $C_{\max}$ (if stage 2's work is complete at its last idle instant, then $C_{\max}$ is at most the last availability, at most W/k, and we are done). At t_1, work conservation means every available job is already finished at stage 2, so the remaining stage-2 work is at most $W - w(A(t_1))$, where $A(t)$ is the set of jobs available by t and $w(\cdot)$ sums their works; stage 2 is busy at rate k on $[t_1, C_{\max}]$, giving $C_{\max} \le t_1 + (W - w(A(t_1)))/k$.

Since stage 2 idles at t_1 with some unfinished job not yet available, some stage-1 machine is still busy; stage-1 machines have no upstream and are work-conserving, so that machine has processed exactly $t_1 k \le W$. Every still-busy machine i has processed exactly $t_1 k$ units, of which at most $w_{\max}$ sit in its single in-progress job, so its set $F_i(t_1)$ of finished jobs satisfies $w(F_i(t_1)) \ge t_1 k - w_{\max}$ (machines already done satisfy this trivially). A job is available iff finished on all m machines, so by inclusion–exclusion for the work measure,

$$w(A(t_1)) = w\Big(\bigcap_i F_i(t_1)\Big) \;\ge\; \sum_i w(F_i(t_1)) - (m-1)W \;\ge\; m(t_1 k - w_{\max}) - (m-1)W = W - m\,w_{\max}.$$

Set $T^* = (m-1)W/(mk) + w_{\max}/k$. If $t_1 \le T^*$ then $C_{\max} \le t_1 + W/k \le (2 - 1/m)W/k + w_{\max}/k$. If $t_1 \ge T^*$ then $C_{\max} \le t_1(1-m) + (mW + m\,w_{\max})/k$, which is decreasing in t_1 and equals $(2-1/m)W/k + w_{\max}/k$ at $t_1 = T^*$.

Lower bound (rotation construction). Equal works w, $m \mid n$. Let machine i process the jobs in the cyclic rotation of $(1, \dots, n)$ by $(i-1)n/m$ positions. The m rotation offsets are evenly spaced, so job j's positions across the machines form the set $\{(j \bmod (n/m)) + r\,n/m + 1 : r = 0, \dots, m-1\}$; its *worst* position is at least $(m-1)n/m + 1$. Hence no job is available before $T_0 = ((m-1)n/m + 1)\,w/k$, and stage 2—however scheduled—must process all $W = nw$ units after T_0: $C_{\max} \ge T_0 + nw/k = (2 - 1/m)\,nw/k + w/k$. Against the optimum $(n+1)w/k$ the ratio tends to $2 - 1/m$. (For $m \nmid n$, rotating by $\lfloor n/m \rfloor$ degrades the constant by $O(w/k)$.) □

Remark 10 (The dichotomy)**.** *The upper bound of Theorem 9 never refers to the downstream* order*; non-idleness is the only property used. So for makespan the downstream behaviors split into exactly two classes: any work-conserving stage, even one that ignores priorities entirely, is protected at* $2 - 1/m$*; only a work-withholding stage—one that holds capacity idle for its preferred job—reaches the full price of Theorem 7. With two upstream machines the numbers are stark: upstream misalignment alone costs at most* 50%*; downstream withholding doubles it. In operational terms, enforcing a priority order* inside *a downstream stage is precisely a withholding discipline whenever the preferred job has not yet arrived, and it forfeits as much again as all upstream misalignment combined. The resemblance of* $2 - 1/m$ *to Graham's list-scheduling bound (Graham, 1966) is discussed in Section 1; both are busy-period consequences of work conservation alone.*

The sentence to remember. If you see a downstream team idle "because the priority item isn't here yet" while finished work sits in their queue, you are watching the single most expensive behavior this mathematics identifies. Downstream, the fix is not a better order — order doesn't enter the proof at all. The fix is: *stages pull*.

5 The price of local ordering: flowtime

For flowtime we take equal works w (so $W = nw$) and a *non-preemptive* one-job-at-a-time downstream stage; the spacing step below fails under preemptive interleaving, whose constant we conjecture is the same (Section 8). The synchronized flowtime is $\sum_s (s+1)w/k = n(n+3)w/(2k)$ by Theorem 2(b), and it is optimal for equal works: the earliest stage-2 completion is at least $2w/k$ (the first job needs w/k at each stage), and the non-preemptive downstream completes jobs at least w/k apart, so every schedule has $C_{(s)} \geq (s+1)w/k$.

Theorem 11 (Full anarchy triples flowtime, exactly)**.** *Over team-sequential schedules with a non-preemptive downstream, for equal works,*

$$\sup \frac{\sum_j C_j}{n(n+3)w/(2k)} \;=\; \frac{3n+1}{n+3} \;\longrightarrow\; 3,$$

the supremum over upstream anarchy with a withholding downstream, attained for every n *when* $m \geq 2$*.*

Proof. Upper bound. Stage 2 processes one job at a time, each taking w/k, so the sorted completions satisfy $C_{(s+1)} \geq C_{(s)} + w/k$; with $C_{(n)} = C_{\max} \leq 2nw/k$ (Theorem 7), $C_{(s)} \leq (n+s)w/k$ and $\sum_j C_j \leq (3n^2 + n)\, w/(2k)$ for *every* team-sequential schedule. The ratio to $n(n+3)w/(2k)$ is $(3n+1)/(n+3)$.

Lower bound. In the construction of Theorem 7's lower bound, stage 2 idles until nw/k and then completes one job per w/k: completions are exactly $(n+s)w/k$ for $s = 1, \ldots, n$, meeting the upper bound term by term. □

From dates to waits. Makespan only counts the last landing; flowtime counts everyone's. Under the shared list, landings form a steady staircase from early on: one initiative lands roughly every w/k. Under full anarchy the staircase is *shifted*: nothing lands during the first half (the withholding stall), then the same staircase plays out late. Every initiative's wait absorbs the stall, so the *sum* of waits is hit roughly three times harder than the final date. The exact finite formula $(3n+1)/(n+3)$ is one of the numbers the brute-force enumeration later reproduces to the decimal.

The "non-preemptive" fine print: the staircase argument needs the gate to finish one item before starting another. If it may pause items midway the bookkeeping changes; the constant is conjectured to survive, and the verification model is non-preemptive, so theory and experiment are compared like for like.

Theorem 12 (Work conservation caps the flowtime price at $3 - 2/m$)**.** *Under the hypotheses of Theorem 11 but with any work-conserving downstream, for equal works and* $m \mid n$*,*

$$\sum_j C_j \;\leq\; \left(\tfrac{3}{2} - \tfrac{1}{m}\right) \frac{n^2 w}{k} + O(n),$$

and the rotation construction attains $\sum_j C_j = (\tfrac{3}{2} - \tfrac{1}{m})\, n^2 w/k + O(n)$ *under every downstream discipline. The price is exactly* $3 - 2/m$ *asymptotically.*

Proof. Upper bound. By Theorem 9, $C_{(n)} \leq ((2 - 1/m)n + 1)\, w/k$; spacing gives $C_{(s)} \leq ((1 - 1/m)n + s + 1)\, w/k$; summing yields the claim.

Lower bound. In the rotation instance the m jobs with residue c become available exactly at slot $(m-1)n/m + c + 1$, so m new jobs arrive

Same dichotomy, second currency. The pattern is worth tabulating:

	gate pulls	gate withholds
portfolio date	$2 - 1/m$	2
sum of waits	$3 - 2/m$	3

In both currencies, the withholding gate gives back exactly as much as all the upstream misalignment cost in the first place. The remedy ranking follows directly: first make every stage pull (cheap, halves the damage), then align the upstream order (eliminates the rest).

per slot from T_0 on while stage 2 completes one per slot: it never starves after T_0, and completions are exactly $T_0 + s\,w/k$, summing to $nT_0 + n(n+1)w/(2k) = (\frac{3}{2} - \frac{1}{m})n^2 w/k + O(n)$. Since no job is available before T_0, no downstream discipline does better than these completions term by term. □

The pattern of Remark 10 thus persists in the flowtime currency—$3 - 2/m$ for any work-conserving downstream versus 3 for a withholding one—with the same interpretation: withholding forfeits as much again as all upstream misalignment.

6 No ceiling for weighted completion time

Proposition 13 (The weighted price is unbounded)**.** *For every B there is an instance and an unsynchronized team-sequential schedule whose weighted completion time exceeds B times the optimum.*

Proof. Put value v on a single job J of work w and negligible value on the others. The synchronized schedule with J first completes it at $2w/k$, so the optimum is at most $2vw/k + o(v)$. The construction of Theorem 7 applied to J completes it at $2W/k$, giving weighted objective at least $2vW/k$. The ratio is $W/w + o(1)$, unbounded as the number of jobs grows. □

Schedule damage is capped at a factor of two (Theorem 7); value damage has no cap, and it grows exactly with the concentration of value. Combined with Corollary 4 this separates the two roles of the shared order: *any* shared order is optimal for the date all work lands; a *correct* (value-aware) shared order is what protects when value lands.

Where there is no safety net. Suppose one small initiative carries most of the portfolio's value. Done first, it lands almost immediately. Buried by one team's local list and waited-for by the gate, it lands *last* — and the value delay scales with the *whole portfolio's* size, not the item's. Bigger portfolio, worse multiplier, no cap. In the enumeration, a three-initiative portfolio with value split 100:1:1 already shows worst-case value delay of $+475\%$.

The division of labor between sharing and ranking: *that* the list is shared protects the schedule (capped at $2\times$, and the date is optimal under any shared order); *how* the list is ranked protects the value (uncapped exposure). Sharing is free insurance; ranking by value is where judgment earns its keep.

7 Computational verification

The constants above are exact at finite sizes, so they can be confronted with exhaustive enumeration. We use the 39-configuration instance family of Bowman (2026a) ($n \le 5$ jobs, $m \in \{2, 3\}$ stage-1 machines, $k \in \{2, 3\}$ workers, symmetric, asymmetric, value-weighted and machine-heterogeneous works; about 4.0×10^6 schedules in total), with every combined schedule classified as synchronized or unsynchronized and, within the unsynchronized population, the downstream order classified as availability-consistent (work-conserving in the integer model) or re-ordered (withholding). The integer model forces balanced divisions exactly when $w_j = k$; in those configurations the fluid constants must be—and are—attained exactly.

Table 1: Worst-case makespan price (per cent over the optimum) of the unsynchronized population, split by downstream discipline, on the division-forced configurations, against the finite-n theory of Theorems 7 and 9.

Configuration	Work-conserving		Withholding	
	observed	theory	observed	theory
$n = 3$, $m = 2$	$+25.0\%$	$5/4$	$+50.0\%$	$6/4$
$n = 4$, $m = 2$	$+40.0\%$	$7/5$	$+60.0\%$	$8/5$
$n = 5$, $m = 2$	$+33.3\%$	$8/6$	$+66.7\%$	$10/6$
$n = 3$, $m = 3$	$+50.0\%$	coincide	$+50.0\%$	$6/4$
$n = 4$, $m = 3$	$+40.0\%$	$7/5$	$+60.0\%$	$8/5$
$n = 2$, any m	$+33.3\%$	$4/3$	class empty (forced)	

Beyond the exact agreement of every entry, two structural predictions are reproduced: at $n = 3$, $m = 3$ the two downstream classes coincide,

How to read the table. Each row is a small organization size where *every possible schedule* was enumerated — millions in total — and the worst uncoordinated outcome was found by inspection, separately for gates that pull and gates that withhold. "Theory" is the fraction the theorems predict. Every cell matches to the decimal.

The last two rows are the interesting kind of check: the theory doesn't just predict numbers, it predicts *structure*. With at least as many teams as initiatives, the rotation trick defeats even a perfect gate — so the two columns must *merge* (they do). With only two initiatives, both arrive at the same moment — so a withholding gate *cannot exist as a separate behavior* (the class is empty, as forced). Predictions of this kind are where a wrong theory would have nowhere to hide. The discipline of checking them is a lesson this research program learned the hard way: an earlier conjecture in this series survived twenty million tests and was still wrong, because the tests sampled a blind spot — see the correction in Bowman (2026b).

as they must once $m \geq n$ (the rotation construction then defeats every downstream discipline); and at $n = 2$ the withholding class is empty, as it must be (availabilities tie, so every downstream order is availability-consistent). The flowtime worst cases similarly match Theorems 11 and 12 exactly where divisions are forced ($+100.0\% = (3n+1)/(n+3)$ at $n = 5$; $+50.0\%$ for the work-conserving class, the rotation value). Configurations with division freedom exceed the balanced-split constants in both classes, quantifying how division badness stacks on top of ordering badness.

Averages tell the operational story: across the full family, the mean makespan price of an unsynchronized schedule is 39.6% (per-configuration means 20.0–63.2%), the mean flowtime price 60.6%, and the mean weighted price 69.2% (worst +475.5%, under value skew 100:1:1); on average across configurations, more than 99.5% of unsynchronized schedules are strictly later than the optimum on every objective (no configuration falls below 95.6%). Within the unsynchronized population, conditioning on an availability-consistent downstream roughly halves the mean price (e.g. from +53.1% to +26.3% at $n = 5$, $m = 2$)—work conservation alone recovers about half the damage of upstream misalignment, before any upstream coordination.

The numbers leadership should retain. A typical uncoordinated run — not a worst case, the *average* over every possible uncoordinated schedule — lands the portfolio about 40% later, the average initiative about 60% later, and the value about 70% later than the shared list. More than 99.5% of uncoordinated schedules are strictly worse: this is not a coin flip you might win. And the cheapest intervention is confirmed empirically: just making the gate pull — touching nothing upstream — cut the average damage roughly in half.

The two bracketing results are formalized in Lean 4 over the concrete machine model of Bowman (2026a): the universal lower bound $k\,C_{\max} \geq W + w_M$ for every job M (Theorem 2(a)), with stage 2 abstracted so that any physical execution satisfies the axioms; and the universal upper bound at the release-time interface behind Theorem 7, together with computed witnesses for the smallest anti-aligned instance ($2W$ versus $W + w_{\max}$). Both developments contain no unproven steps. Code, enumeration outputs, and the Lean development are in the archived artifact (see Declarations).

8 Discussion

The operating model this licenses, in three steps: (1) every stage pulls — never idle while ready work waits (halves the damage, costs nothing); (2) one shared list at the source — any shared list fixes the portfolio date, the value-ranked one protects the value; (3) rank by value and re-rank only on real information. Each step's worth is now a theorem with a constant attached, not a slogan.

Together with Bowman (2026a,b), the results complete a two-sided account of the shared order in this model. The dominance theorem says adopting it is free; this paper says declining it costs 40% on average in the exhaustive family, at most a factor of two on the schedule (exactly), without limit on concentrated value—and that the two halves of the price have different owners. Upstream misalignment is worth $2 - 1/m$; everything beyond is the downstream stage's choice to hold capacity idle for a preferred job. For practice the asymmetry is the message: a downstream stage that simply keeps pulling whatever has arrived is protected by Theorem 9 no matter how badly it orders; the behavior to eliminate is waiting—which is what enforcing a priority list inside a stage operationally means whenever the list's next item has not arrived. The stochastic results add that none of this depends on knowing the work: a fixed shared order is exactly optimal for the portfolio date under any randomness (Corollary 4), while protecting *value* requires the order to be right (Proposition 13)—sharedness is free insurance, correctness buys value.

Open problems. (i) The flowtime spacing arguments require a non-preemptive downstream; we conjecture the constants 3 and $3 - 2/m$ survive preemption. (ii) The flowtime constants are proved for equal works; the general-works analogues should follow the same pattern. (iii) The expected price under *random* (rather than adversarial) per-machine orders is strictly positive and empirically large (39.6% mean above); its exact asymptotics are open. (iv) Whether the value-ordered fixed permutation is expectation-optimal for weighted completion time under exchangeable works (the WSEPT analogue) is open. (v) The online version—jobs arriving over time, the shared order maintained by preemptive priority—is the natural continuation and the model closest

What's deliberately left open. The largest of these is (v): everything in this paper schedules a *planned batch* of work. The live version — work arriving continuously, the shared list maintained by letting higher-priority work preempt — is how organizations actually run, and proving the same guarantees there is the next paper-sized question in this series.

to the motivating applications.

Reproducibility. Everything quantitative in this paper can be regenerated from the public artifact: the enumeration suite reproduces every table entry, and the Lean development re-verifies the two bracketing theorems mechanically on any machine.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. The sole author conceived the work, developed the proofs, and wrote the manuscript.

Data availability. The enumeration outputs supporting Section 7 are available in the repository below.

Code availability. The simulation code, exhaustive-enumeration suite, and the Lean 4 formalization are available at `https://github.com/ebowman/synchronization-paper-artifact` and archived at `https://doi.org/10.5281/zenodo.20075388`.

Generative AI disclosure. AI-assisted tooling (Anthropic Claude) was used for literature search, drafting support, simulation code, and proof checking; all mathematical content was verified by the author, by exhaustive computational enumeration, and—where stated—by the Lean 4 proof assistant.

References

E. Bowman. Permutation-schedule dominance for two-stage assembly flow shops with divisible work. Submitted to *Journal of Scheduling*; preprint doi:10.5281/zenodo.20075388, 2026a.

E. Bowman. The boundary of synchronization dominance: A depth characterization for divisible-work scheduling on DAGs. Submitted to *Journal of Scheduling*; preprint doi:10.5281/zenodo.20075388, 2026b.

M. Drozdowski. *Scheduling for Parallel Processing*. Springer, London, 2009.

R. L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.

R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5:287–326, 1979.

A. M. A. Hariri and C. N. Potts. A branch and bound algorithm for the two-stage assembly scheduling problem. *European Journal of Operational Research*, 103(3):547–556, 1997.

E. Koutsoupias and C. H. Papadimitriou. Worst-case equilibria. In *STACS 99*, volume 1563 of *Lecture Notes in Computer Science*, pages 404–413. Springer, Berlin, 1999.

C.-Y. Lee, T. C. E. Cheng, and B. M. T. Lin. Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. *Management Science*, 39(5):616–625, 1993.

M. L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 5th edition, 2016.

C. N. Potts, S. V. Sevast'janov, V. A. Strusevich, L. N. Van Wassenhove, and C. M. Zwaneveld. The two-stage assembly scheduling problem: Complexity and approximation. *Operations Research*, 43(2):346–355, 1995.

M. H. Rothkopf. Scheduling with random service times. *Management Science*, 12(9):707–713, 1966.

W. E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66, 1956.